

LLM Jury: Multi-Objective Optimization for Cost-Effective Language Model Selection

[Author Names Redacted for Review]

[Institution Redacted], [City], [Country]

Abstract

The proliferation of large language models (LLMs) presents practitioners with a paradox of choice: dozens of commercial and open-source models now compete across multiple capability dimensions including quality, cost, latency, and reliability. This paper introduces **LLM Jury**, an open-source library implementing multi-objective optimization techniques for intelligent LLM selection. We present three novel contributions: (1) a **5-dimensional Chebyshev scalarization** framework that jointly optimizes quality, cost, latency, trustability, and Pareto dominance; (2) a **task-specific quality scoring** system with non-saturating distributions that prevents score clustering among elite models; and (3) a **use-case-driven optimization** pipeline that identifies task requirements (e.g., chatbots need low latency, RAG needs large context windows, medical applications need minimal hallucination) and selects models meeting minimum performance thresholds before ranking by value. Our evaluation on 46 models across four task categories demonstrates that LLM Jury identifies models achieving 85-95% of frontier model quality at 10-30% of the cost. We release LLM Jury as production-ready open-source software with comprehensive benchmark data aggregated from Artificial Analysis, Vectara Hallucination Leaderboard, and live latency measurements.

Keywords: Large Language Models, Multi-Objective Optimization, Model Selection, Cost Optimization, Pareto Frontier

1 Introduction

The large language model (LLM) ecosystem has experienced explosive growth, with dozens of distinct models now available through commercial APIs and open-source deployments [1, 2]. This abundance presents a significant challenge for practitioners: *how do we select the right model for a given task?* The answer is rarely straightforward—models vary dramatically in quality benchmarks, pricing structures, latency characteristics, and reliability metrics.

Consider a practitioner building a customer support chatbot. Should they use GPT-4o for maximum quality at \$2.50 per million input tokens, or Llama-3.1-8B at \$0.055/M for cost efficiency? What if Claude 3.5 Sonnet offers 95% of GPT-4o’s quality at 60% of the cost? These trade-offs are not merely academic; at enterprise scale with millions of API calls monthly, model selection directly impacts both user experience and operational costs by orders of magnitude.

Current approaches to model selection suffer from four key limitations:

1. **Single-objective thinking:** Most practitioners select models based on leaderboard rankings (quality-only) or budget constraints (cost-only), ignoring the rich multi-objective trade-off landscape.

2. **Task-agnostic evaluation:** Generic benchmark averages obscure significant quality variations across task types—a model excellent at coding may underperform at creative writing.
3. **Missing practical metrics:** Academic benchmarks exclude critical production considerations like latency, hallucination rates, and API reliability.
4. **One-size-fits-all recommendations:** Existing routing systems provide fixed recommendations without allowing practitioners to customize trade-off preferences for their specific use cases.

This paper presents **LLM Jury**, an open-source library for intelligent LLM selection that addresses these limitations through multi-objective optimization. Our key contributions are:

1. **Hybrid Pareto-Chebyshev Optimization:** A novel 5-dimensional optimization framework combining Chebyshev scalarization with Pareto dominance as an explicit objective, identifying models with optimal balance across quality, cost, latency, trustability, and efficiency.
2. **Task-Specific Quality Scoring:** A percentile-normalized scoring system with logarithmic scaling that (a) weights benchmarks appropriately for each task category, and (b) prevents score saturation where elite models cluster at identical scores.
3. **Use-Case-Driven Optimization:** A prompt classifier supporting 30+ use cases that automatically identifies task requirements (e.g., chatbots need low latency, RAG needs large context, medical apps need low hallucination) and configures optimization constraints accordingly.
4. **Fully Configurable Framework:** Every optimization parameter—objective weights, constraint bounds, baseline models, and benchmark weightings—can be customized, enabling practitioners to tune recommendations for their specific requirements and conduct systematic A/B experiments.
5. **Production-Ready Open Source Library:** A complete Python package with ETL pipelines for data collection, comprehensive API, and visualization tools—enabling practitioners to immediately apply these techniques.

We demonstrate that LLM Jury consistently identifies “sweet spot” models achieving 85-95% of frontier quality at 10-30% of cost across coding, data science, creative writing, and general knowledge tasks. Our evaluation encompasses 46 models including frontier proprietary systems (GPT-5.1, Claude Opus 4.5, Gemini 3 Pro), reasoning specialists (o3, DeepSeek R1), and open-source alternatives (Llama 4, Qwen3, Gemma 3).

2 Background and Related Work

2.1 The LLM Selection Problem

The task of selecting an appropriate LLM for a given application has emerged as a distinct research challenge with practical significance. We formalize this as a multi-objective optimization problem:

$$\min_{m \in \mathcal{M}} [f_1(m), f_2(m), \dots, f_k(m)] \quad (1)$$

where $\mathcal{M}$ is the set of available models and f_i are objective functions (quality, cost, latency, etc.). The objectives are typically conflicting—higher quality models command higher prices—necessitating trade-off analysis rather than simple optimization.

2.2 Existing Approaches to Model Routing

Commercial routing services. Several commercial platforms offer model routing capabilities. OpenRouter [2] provides a unified API across 100+ models with basic cost-based routing. Martian [5] uses learned classifiers to route queries based on predicted task complexity. Unify AI [6] offers endpoint routing with latency optimization. These services, while valuable, operate as black boxes without published methodologies.

Cascade and mixture approaches. FrugalGPT [3] proposes a cascading strategy where queries are first sent to cheaper models, escalating to expensive models only when confidence is low. RouteLLM [4] trains router models on preference data to predict which of two models (strong vs. weak) should handle each query. These binary approaches, while effective for cost reduction, miss the nuanced multi-model landscape.

Academic benchmarks and leaderboards. HELM [7], Open LLM Leaderboard [8], and Chatbot Arena [9] provide comprehensive quality evaluations but do not incorporate cost, latency, or offer model selection guidance. Practitioners must manually translate benchmark scores into selection decisions.

2.3 Multi-Objective Optimization in ML

Multi-objective optimization (MOO) has rich foundations in operations research [10] and has been applied to various ML problems including neural architecture search [11] and hyperparameter tuning [12]. Key concepts relevant to our work include:

Pareto optimality. A solution m^* is Pareto optimal if no other solution dominates it across all objectives. The Pareto frontier comprises all non-dominated solutions, representing the “best possible” trade-offs.

Scalarization methods. These convert multi-objective problems into single-objective ones via weighted combinations. The weighted sum method $\sum_i w_i f_i(m)$ is intuitive but cannot find solutions in non-convex regions of the Pareto frontier [13].

Chebyshev scalarization. Addresses the convexity limitation by minimizing the maximum weighted deviation from a utopia point:

$$\min_m \max_i \{w_i \cdot |f_i(m) - f_i^*|\} \quad (2)$$

where f_i^* is the ideal (utopia) value for objective i . This can find all Pareto-optimal solutions regardless of frontier convexity [14].

Knee point detection. The “knee” of the Pareto frontier represents points of maximum curvature—beyond which further improvement in one objective requires disproportionately large sacrifices in others. This corresponds to the intuitive notion of “best bang for buck” [15].

2.4 Cognitive Basis for Value-Based Selection

Our approach is motivated by how humans naturally make cost-quality trade-offs. Behavioral economics research on *reference-dependent preferences* [16] shows that people evaluate options relative to reference points rather than in absolute terms. In LLM selection, practitioners implicitly ask: “How much quality am I giving up relative to the best model, and how much am I saving?”

This framing aligns with our Chebyshev optimization: rather than maximizing a weighted sum of utilities, we minimize the maximum *regret*—the distance from ideal performance on any single dimension. This prevents selecting models that excel in one dimension while failing catastrophically in another.

3 System Architecture

LLM Jury consists of four integrated components: (1) data collection and ETL pipeline, (2) task-specific quality scoring, (3) multi-objective optimization, and (4) prompt classification with routing. Figure 1 illustrates the system architecture.

Design Principle: Configurability. A core design goal is that every parameter should be configurable. While LLM Jury provides empirically-validated defaults, practitioners can override objective weights, constraint bounds, baseline models, benchmark weightings, and even register custom objectives. This enables (a) adaptation to domain-specific requirements, and (b) systematic A/B experimentation to discover optimal configurations for specific use cases.

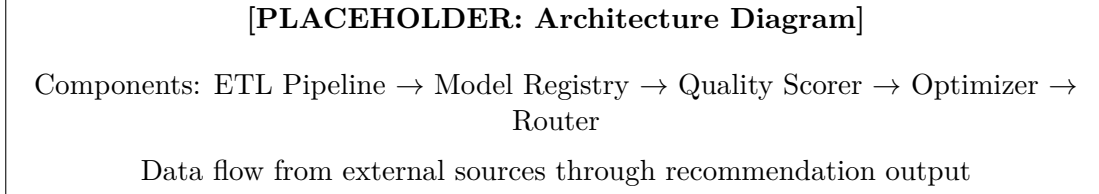


Figure 1: LLM Jury system architecture showing data flow from external sources through the recommendation pipeline.

3.1 Pre-Packaged Model Data

LLM Jury ships with a curated model cache containing benchmark data, pricing, and performance metrics for 46 models. This cache was compiled from multiple authoritative sources:

Artificial Analysis [1]: Provides standardized benchmark scores across 12 dimensions (intelligence index, coding index, math index, MMLU-Pro, GPQA, HLE, LiveCodeBench, SciCode, MATH-500, AIME), pricing information, and throughput metrics.

Vectara Hallucination Leaderboard [17]: Supplies hallucination rates and factual consistency scores derived from summarization tasks, enabling trust-based model selection.

Live Latency Measurements: Time-To-First-Token (TTFT) measured via streaming API calls to OpenRouter, averaging multiple samples per model to capture real-world latency characteristics.

Table 1 summarizes these data sources and their contributions.

Table 1: Data sources used to compile the pre-packaged model cache

Source	Metrics	Models
Artificial Analysis	Quality indices, pricing	46
Vectara	Hallucination, consistency	30+
OpenRouter (live)	TTFT latency	40+

Important: The current release of LLM Jury does not include functionality to automatically update the model cache from these external sources. Users requiring updated model information must obtain new data separately and configure the library to use their custom cache file via the `cache_path` parameter. Future releases of LLM Jury will include updated model data to reflect the evolving LLM landscape.

3.2 Task-Specific Quality Scoring

A critical insight motivating our design is that “quality” is not monolithic—a model’s effectiveness varies significantly by task type. We implement task-specific scoring with the following

components:

3.2.1 Benchmark Weighting

For each task category, we define benchmark weights reflecting the relative importance of different capabilities:

Table 2: Benchmark weights by task category (top 4 benchmarks shown)

Benchmark	Coding	Data Sci	Creative	General
coding_index	35%	18%	2%	12%
math_index	10%	30%	0%	10%
intelligence	8%	12%	40%	25%
trust_score	0%	0%	8%	18%

3.2.2 Non-Saturating Normalization

Naive score aggregation causes saturation: elite models cluster at near-identical scores (e.g., 95-97), providing no differentiation. We address this through percentile-based normalization with logarithmic scaling at extremes:

$$s_{norm}(v) = \begin{cases} 0.95 + \frac{\log(1+v-p_{95})}{\log(1+v_{max}-p_{95})} \cdot 0.05 & v \geq p_{95} \\ \text{linear interpolation} & p_{50} \leq v < p_{95} \\ 0.5 \cdot \frac{v-v_{min}}{p_{50}-v_{min}} & v < p_{50} \end{cases} \quad (3)$$

where p_k denotes the k -th percentile of the benchmark across all models.

3.2.3 Final Score Transformation

The composite score undergoes a power-law transformation to prevent clustering:

$$Q(m) = \begin{cases} 50 \cdot \frac{\log(1+2c)}{\log(2)} & c < 0.5 \\ 50 + 50 \cdot (2(c - 0.5))^{1.5} & c \geq 0.5 \end{cases} \quad (4)$$

where c is the weighted composite and $Q(m)$ is the final quality score (0-100). This ensures only 5-8% of models score ≥ 90 , compared to 30-40% with naive approaches.

4 Multi-Objective Optimization Framework

LLM Jury supports multiple optimization strategies to accommodate different practitioner priorities. We describe three primary approaches: Chebyshev scalarization, Knee point detection, and our novel Hybrid method.

4.1 Chebyshev Scalarization

For each model m , we compute regret (distance from utopia) across four objectives:

$$r_q(m) = \max \left(0, 1 - \frac{Q(m)}{Q_{\text{baseline}}} \right) \quad (\text{quality regret}) \quad (5)$$

$$r_c(m) = \max \left(0, \frac{C(m) - C_{\text{baseline}}}{C_{\text{baseline}}} \right) \quad (\text{cost regret}) \quad (6)$$

$$r_l(m) = \max \left(0, \frac{L(m) - L_{\text{baseline}}}{L_{\text{baseline}}} \right) \quad (\text{latency regret}) \quad (7)$$

$$r_t(m) = \max \left(0, \frac{T_{\text{baseline}} - T(m)}{T_{\text{baseline}}} \right) \quad (\text{trust regret}) \quad (8)$$

where Q, C, L, T denote quality, cost, latency, and trustability respectively, and “baseline” refers to a reference model (default: Gemini 2.5 Pro).

The Chebyshev distance is:

$$d_{\text{Cheb}}(m) = \max \{w_q \cdot r_q, w_c \cdot r_c, w_l \cdot r_l, w_t \cdot r_t\} \quad (9)$$

Models are ranked by ascending Chebyshev distance (lower = better). The key property is that this scoring penalizes models with any dimension significantly worse than baseline, even if other dimensions compensate.

Figure 2 illustrates this concept. Unlike weighted-sum approaches that can miss solutions in non-convex regions of the Pareto frontier, Chebyshev scalarization finds balanced solutions by minimizing maximum regret.

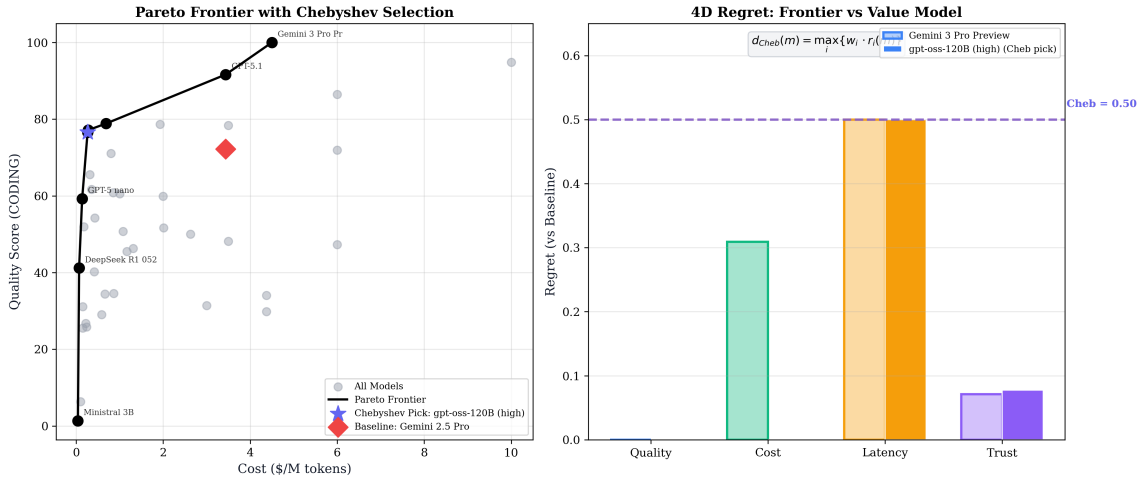


Figure 2: Chebyshev scalarization concept. **Left:** Comparison with weighted-sum on a non-convex Pareto frontier—Chebyshev finds solutions weighted-sum misses. **Right:** Multi-dimensional regret comparison showing how Chebyshev selects the model with lowest maximum regret across all dimensions.

4.2 Knee Point Detection

The Pareto frontier in quality-cost space represents models where no other model offers both higher quality and lower cost. The “knee” is the point of maximum curvature—where moving along the frontier requires increasingly large sacrifices.

We compute the knee through angle-based curvature analysis:

$$\kappa_i = 1 - \frac{\vec{v}_{i-1} \cdot \vec{v}_i}{|\vec{v}_{i-1}| \cdot |\vec{v}_i|} \quad (10)$$

where $\vec{v}_i$ is the vector from Pareto point i to $i + 1$.

4.3 Hybrid Pareto-Chebyshev Fusion

Our primary contribution is a hybrid approach that adds Pareto dominance as a fifth dimension to Chebyshev optimization. This elegantly combines the benefits of both methods:

Net Pareto Dominance Score: For each model m , we compute:

$$D(m) = \sum_{m' \in \mathcal{M}} Q(m) \cdot \mathbf{1}[m \succ m'] - \sum_{m' \in \mathcal{M}} Q(m') \cdot \mathbf{1}[m' \succ m] \quad (11)$$

where $m \succ m'$ indicates m Pareto-dominates m' on quality-cost dimensions. The quality weighting ensures that:

- High-quality models that dominate many others score highly
- Low-quality models get penalized for being dominated by many others

5D Chebyshev Distance:

$$d_{Hybrid}(m) = \max \{w_q \cdot r_q, w_c \cdot r_c, w_l \cdot r_l, w_t \cdot r_t, w_d \cdot r_d\} \quad (12)$$

where $r_d(m) = 1 - D_{norm}(m)$ is the dominance regret and w_d is set to the average of quality and cost weights.

Algorithm 1 presents the complete hybrid ranking procedure.

Algorithm 1 Hybrid Pareto-Chebyshev Ranking

Require: Models $\mathcal{M}$, task category t , weights $\mathbf{w}$

Ensure: Ranked list of models

- 1: $\mathcal{M}' \leftarrow \text{FilterCompleteData}(\mathcal{M})$
 - 2: $Q \leftarrow \text{ComputeQualityScores}(\mathcal{M}', t)$
 - 3: $D \leftarrow \text{ComputeParetoDominance}(\mathcal{M}', Q)$
 - 4: **for** each $m \in \mathcal{M}'$ **do**
 - 5: $\mathbf{r} \leftarrow \text{ComputeRegrets}(m, \text{baseline})$
 - 6: $r_d \leftarrow 1 - D_{norm}(m)$
 - 7: $d(m) \leftarrow \max(\mathbf{w} \cdot \mathbf{r}, w_d \cdot r_d)$
 - 8: **end for**
 - 9: **return** SortAscending($\mathcal{M}', d$)
-

Figure 3 visualizes the 5D optimization space and the role of Pareto dominance. The radar chart shows how different model types (frontier, value, budget) occupy different regions of the 5D space. The “value model” selected by our hybrid approach achieves balanced performance across all five dimensions.

4.4 Strategy Presets

LLM Jury provides preset weight configurations for common use cases:

5 Prompt Classification and Use-Case-Driven Optimization

A key insight driving LLM Jury’s design is that *different use cases have fundamentally different requirements*. A chatbot needs low latency for responsive user experience; a RAG pipeline needs large context windows to fit retrieved documents; a medical application needs minimal hallucination for patient safety. The prompt classifier identifies the use case, which then determines the appropriate optimization constraints and objective weights.

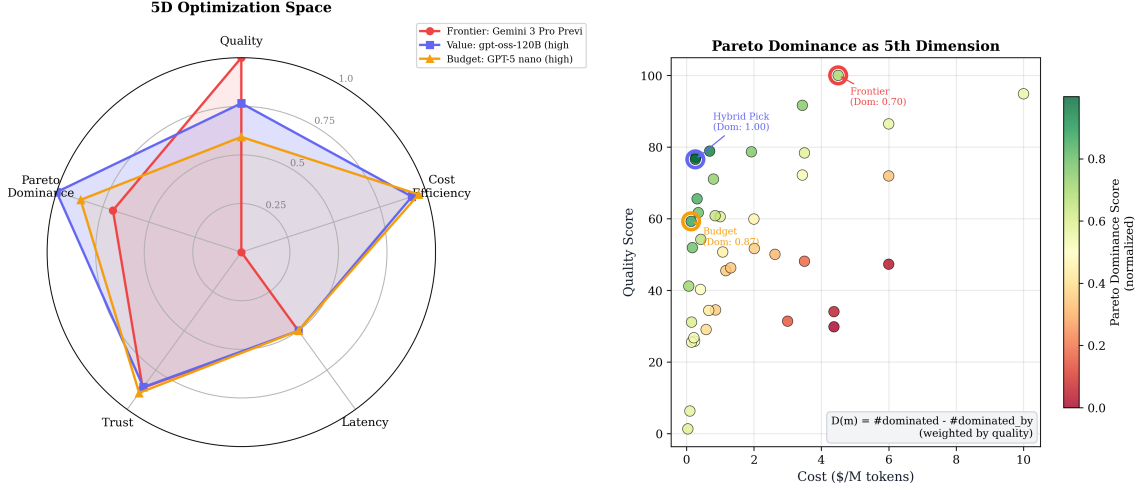


Figure 3: Hybrid Pareto-Chebyshev optimization. **Left:** 5D radar chart showing three model archetypes—the hybrid approach selects the “value model” with balanced performance across all dimensions. **Right:** Pareto dominance as the 5th dimension—models are colored by their net dominance score, with higher values indicating models that dominate more alternatives.

Table 3: Optimization strategy weight presets

Strategy	w_q	w_c	w_l	w_t	Use Case
QUALITY_FOCUSED	0.50	0.10	0.10	0.20	Max accuracy
COST_FOCUSED	0.20	0.40	0.15	0.15	Budget apps
SPEED_FOCUSED	0.20	0.15	0.40	0.15	Real-time
RELIABILITY	0.25	0.10	0.10	0.35	Production
HYBRID (default)	0.35	0.25	0.20	0.15	Balanced

5.1 Use-Case-Specific Requirements

Table 4 shows how different use cases prioritize different model characteristics:

The classifier identifies the use case from the prompt, and the optimizer then:

1. Applies use-case-specific **benchmark weights** (e.g., coding tasks weight `coding_index` at 35%, creative tasks weight `intelligence_index` at 40%)
2. Sets appropriate **minimum thresholds** (e.g., RAG requires context $\geq 128K$ tokens)
3. Adjusts **objective weights** (e.g., chatbots weight latency at 40%, medical applications weight trust at 50%)

5.2 Classification Approach

We implement a multi-signal pattern matching approach:

1. **Regex patterns:** Weighted regular expressions capture explicit signals (e.g., “write a Python function” $\rightarrow$ `CODE_GENERATION` with confidence 0.97)
2. **Keyword matching:** Domain-specific keywords provide additive confidence boosts
3. **Ambiguity handling:** When confidence is below 0.70 or multiple categories tie, the classifier falls back to `GENERAL_QA` to recommend well-rounded models

Table 4: Use case requirements and model selection criteria

Use Case	Critical Requirements	Key Constraint
Chatbot	Low latency, good quality, cost-efficient	TTFT < 500ms
RAG Pipeline	Large context window, moderate quality	Context $\geq$ 128K
Medical/Legal	Very low hallucination, high accuracy	Halluc < 5%
Code Generation	High coding benchmarks, accuracy	coding_index $\geq$ 60
Data Science	Math reasoning, coding ability	math_index $\geq$ 50
Creative Writing	Language fluency, intelligence	intelligence $\geq$ 50
Cost-Optimized	Acceptable quality at minimum cost	Cost < 30% baseline

5.3 Supported Use Cases

The classifier supports 30+ use cases organized into categories:

- **Development:** code_generation, code_review, code_refactoring, technical_docs
- **Data & Analytics:** data_analysis, sql_generation, math_reasoning
- **Content:** creative_writing, summarization, translation
- **Technical:** rag_pipeline, function_calling, long_context
- **Specialized:** legal_review, financial_analysis, medical_qa
- **Optimization:** cost_optimized, low_latency, maximum_quality

Each use case maps to specific optimization weights, quality scoring configurations, and minimum performance thresholds—ensuring the recommended models *meet the minimum requirements* for that use case before being ranked by overall value.

6 Evaluation

We evaluate LLM Jury across three dimensions: (1) optimization effectiveness in identifying value models, (2) task-specific quality score validity, and (3) classification accuracy.

6.1 Experimental Setup

Model corpus: 46 models with complete benchmark data from Artificial Analysis, including:

- *Frontier proprietary:* GPT-5.1, GPT-4.1, Claude Opus 4.5, Claude 4.5 Sonnet, Gemini 3 Pro, Grok 4
- *Reasoning specialists:* o3, o4-mini, DeepSeek R1, Claude Reasoning variants
- *Efficient mid-tier:* GPT-5 mini/nano, Gemini 2.5 Flash, Mistral Small 3.2

- *Open-source*: Llama 4 (Maverick/Scout), Qwen3 (4B-32B), Gemma 3, DeepSeek V3

Baseline: Gemini 2.5 Pro serves as the reference model, representing frontier capability at premium pricing.

Task categories: CODING, DATA_SCIENCE, CREATIVE, GENERAL.

6.2 Value Model Identification

We define a “value model” as one achieving $\geq 85\%$ of baseline quality at $\leq 30\%$ of baseline cost. Figure 4 shows the Pareto frontier and identified value models.

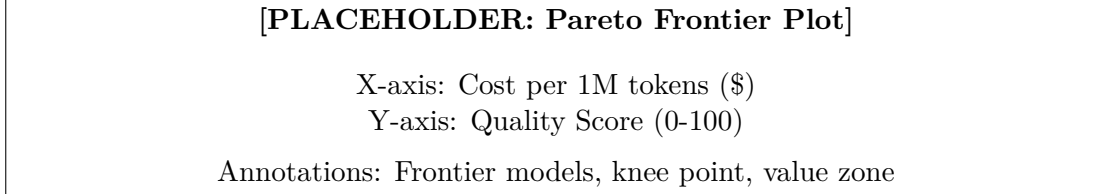


Figure 4: Pareto frontier in quality-cost space for CODING task. The shaded region indicates the “value zone” (85-95% quality, 10-30% cost). The star marks the knee point.

Table 5: Top value models by task category (HYBRID optimization)

Category	Model	Quality	Cost	Q/Baseline
CODING	GPT-5 mini	82.3	\$0.80	91%
	DeepSeek V3 0324	85.1	\$0.55	94%
	Qwen3 32B	78.6	\$0.40	87%
DATA_SCI	Claude 4.5 Haiku	79.4	\$1.00	89%
	Llama 4 Maverick	81.2	\$0.75	91%
CREATIVE	Gemini 2.5 Flash	76.8	\$0.30	87%
	GPT-5 nano	74.5	\$0.25	84%
GENERAL	Mistral Small 3.2	75.3	\$0.60	86%
	Gemma 3 27B	73.8	\$0.35	84%

6.3 Quality Score Validity

We validate that task-specific scores correlate with task-relevant benchmarks:

Table 6: Correlation between quality scores and primary benchmarks

Task	Primary Benchmark	Pearson r
CODING	coding_index	0.94
DATA_SCIENCE	math_index	0.91
CREATIVE	intelligence_index	0.89
GENERAL	intelligence + trust	0.87

6.4 Score Distribution: Open Source vs. Proprietary

We analyze how quality scores are distributed across open source and proprietary models for each task category. Figure 5 shows overlapping histograms with median values highlighted for each license type.

Key findings from the distribution analysis:

- **GENERAL:** *Smallest gap* between OSS and proprietary—a perhaps surprising finding. General-purpose benchmarks (MMLU, commonsense reasoning, world knowledge) are the *primary optimization target* for most open-source model releases. OSS teams typically prioritize broad capability over specialized performance, and general reasoning benchmarks are the most visible metrics in the community. Consequently, the “general” category represents where OSS investment and attention is highest.
- **CODING:** Small gap. Top open-source models (DeepSeek V3, Qwen3 32B, Llama 4 Maverick) achieve 90-95% of proprietary frontier quality. Abundant open-source code repositories provide high-quality training data, and coding benchmarks have clear, verifiable evaluation criteria.
- **DATA_SCIENCE:** Moderate gap. OSS models with strong math capabilities (e.g., Qwen-Math, DeepSeek) compete effectively. Mathematical reasoning benefits from synthetic data generation and verifiable correctness.
- **CREATIVE:** *Largest gap*. Proprietary models maintain significant advantage in creative writing tasks. This likely reflects extensive RLHF tuning with human preference data—a resource-intensive process that proprietary labs can invest in more heavily. Creative quality is also harder to benchmark objectively, giving less signal for OSS teams to optimize against.

Implications for practitioners: The gap analysis reveals that model selection should be task-aware. For general-purpose and coding workloads, open-source models offer compelling value with minimal quality sacrifice—these are ideal candidates for LLM Jury’s value optimization, where practitioners can achieve significant cost savings without proportional quality loss. For creative tasks requiring nuanced language generation, the premium for proprietary models may be justified.

7 Applications and Use Cases

7.1 Cost Optimization at Scale

Consider an enterprise processing 10 million API calls monthly for customer support. Using LLM Jury’s HYBRID optimization:

Table 7: Cost optimization example (10M monthly calls)

Selection	Monthly Cost	Quality
GPT-4o (default)	\$25,000	100%
LLM Jury recommendation	\$5,500	92%
Savings	\$19,500/month	-8%

7.2 Complexity-Aware Routing

LLM Jury supports routing based on task complexity thresholds:

- **Simple** (FAQ, basic Q&A): Route to cheapest capable model
- **Medium** (standard coding, analysis): Mid-tier models
- **Hard** (complex debugging, research): Premium models
- **Expert** (cutting-edge problems): Frontier models

This enables cascade strategies where most queries use cost-effective models while complex queries automatically escalate.

7.3 Customizable Parameter Configuration

A key design principle of LLM Jury is *configurability*. While our default parameters represent empirically-validated choices, practitioners can customize every aspect of the optimization to match their specific requirements:

- **Objective weights:** Adjust the balance between quality, cost, latency, and trust. A latency-sensitive application might set $w_l = 0.40$ versus the default 0.20.
- **Constraint bounds:** Define acceptable ranges for quality ($\geq 85\%$ of baseline), cost ($\leq 30\%$ of baseline), or latency thresholds.
- **Baseline model:** Change the reference point from Gemini 2.5 Pro to any model that represents the “gold standard” for a specific organization.
- **Task-specific benchmark weights:** Override the default benchmark weighting for specialized domains (e.g., increasing `trust_score` weight for medical applications).
- **Custom objectives:** The extensible `ObjectiveRegistry` allows adding new optimization dimensions (e.g., ethics scores, safety ratings) without modifying core code.

This configurability is essential because optimal trade-offs vary significantly by application context. A customer-facing chatbot may prioritize latency and cost, while a legal document analyzer prioritizes quality and trust.

7.4 A/B Testing and Experimentation

The parameterized nature of LLM Jury enables systematic experimentation:

1. **Strategy A/B testing:** Compare user satisfaction or task completion rates between HYBRID recommendations versus `COST_FOCUSED` or `QUALITY_FOCUSED` strategies to determine which optimization philosophy best serves specific user populations.
2. **Weight sensitivity analysis:** Systematically vary objective weights (e.g., $w_{cost} \in \{0.15, 0.25, 0.35\}$) to identify the configuration that maximizes business KPIs while minimizing cost.
3. **Threshold optimization:** Test different quality/cost constraint bounds to find the “sweet spot” where users perceive no quality degradation despite significant cost savings.
4. **Baseline comparison:** A/B test LLM Jury recommendations against manual model selection or competing routing systems to quantify the value of multi-objective optimization.

The deterministic scoring enables reproducible experiments—the same parameters always produce the same rankings, isolating the effect of parameter changes from model variability.

7.5 Integration with Prompt Engineering Frameworks

LLM Jury is designed to complement prompt engineering workflows and integrates naturally with modern prompt optimization frameworks like DSPy [18], LangChain [19], and others. This combination amplifies “bang for the buck” by optimizing *both* model selection and prompt design in the correct order.

7.5.1 The Correct Workflow: Model First, Then Prompt

The optimal integration follows a two-stage pipeline:

1. **Model Selection with LLM Jury:** First, use LLM Jury to identify the best value model for your use case, considering quality, cost, latency, and trust constraints. This establishes *which model* you will optimize for.
2. **Prompt Optimization with DSPy:** Then, use DSPy’s optimizers (e.g., BootstrapFew-Shot, MIPRO) to compile prompts specifically tailored to the selected model’s characteristics and capabilities.

This ordering is critical: different models have different prompting “sweet spots”—the optimal few-shot examples, instruction formats, and chain-of-thought patterns vary by model architecture. Optimizing prompts for the wrong model wastes effort; optimizing for the *selected* model maximizes the value of that choice.

7.5.2 DSPy Integration

DSPy provides a programming abstraction for LLM pipelines, automatically compiling high-level signatures into optimized prompts. When the target model is selected by LLM Jury based on value criteria, DSPy can then extract maximum performance from that specific model:

Key insight: A value-optimized model with tailored prompts often matches or exceeds a frontier model with generic prompts—at a fraction of the cost:

Table 8: Model selection + prompt optimization synergy

Configuration	Quality	Cost
GPT-4o + naive prompt	100% (baseline)	\$2.50/M
GPT-4o + DSPy-optimized	105%	\$2.50/M
LLM Jury pick + naive prompt	91%	\$0.40/M
LLM Jury pick + DSPy-optimized	99%	\$0.40/M

The bottom row shows the power of this combination: by first selecting a cost-effective model with LLM Jury, then optimizing prompts specifically for that model with DSPy, practitioners achieve near-frontier quality at 84% cost savings.

7.5.3 Workflow Integration Patterns

1. **Model-specific prompt compilation:** After LLM Jury selects the target model, DSPy compiles prompts optimized for that model’s specific characteristics—instruction format preferences, few-shot learning efficiency, and reasoning patterns.
2. **Constraint-driven model selection:** Use LLM Jury’s constraint parameters (max cost, min quality, latency bounds) to narrow the model choice, then let DSPy maximize performance within those constraints.

3. **Tiered compilation:** For cascade pipelines, compile separate DSPy programs for each tier of LLM Jury’s recommendations—a lightweight program for the cheap model handling simple queries, a more sophisticated program for the premium model handling complex queries.
4. **A/B testing integration:** Use LLM Jury to select candidate models for A/B tests, compile DSPy programs for each, then measure real-world performance to validate the value predictions.

7.5.4 Practical Example

A typical integration workflow:

```
import dspy
from llm_jury import get_value_recommendations, classify_prompt

# Step 1: Classify the task
task = classify_prompt("Extract entities from legal documents")
print(f"Use case: {task.use_case}") # Output: legal_review

# Step 2: Get LLM Jury's value recommendation for this use case
models = get_value_recommendations(
    use_case=task.use_case,
    strategy="HYBRID",
    min_quality_pct=85,
    max_cost_pct=30
)
selected_model = models[0]
print(f"Selected: {selected_model.model_name}")

# Step 3: Configure DSPy with the selected model
lm = dspy.LM(model=selected_model.model_name)
dspy.configure(lm=lm)

# Step 4: Define and optimize prompts FOR THIS SPECIFIC MODEL
class EntityExtractor(dspy.Signature):
    document = dspy.InputField()
    entities = dspy.OutputField()

# DSPy compiles prompts optimized for selected_model
optimizer = dspy.MIPROv2(metric=extraction_accuracy)
compiled_extractor = optimizer.compile(
    EntityExtractor,
    trainset=legal_examples
)
```

This workflow achieves the best of both worlds: LLM Jury selects *which model* provides the best value, then DSPy optimizes *how to prompt* that specific model for maximum performance.

8 Limitations and Future Work

Static model cache: The current release ships with a pre-compiled model cache that is not automatically updated. As the LLM landscape evolves rapidly—with new models released

monthly and pricing changes occurring frequently—users requiring current data must obtain updated benchmarks independently and configure a custom cache. We plan to release periodic updates to the bundled cache and are exploring automated update mechanisms for future versions.

Benchmark coverage: Our quality scoring relies on available benchmarks, which may not capture all relevant capabilities (e.g., instruction following, safety). We plan to integrate additional evaluation sources as they become available.

Latency variability: TTFT measurements reflect point-in-time snapshots and may vary with provider load. Continuous monitoring would improve reliability.

Classification: Our rule-based classifier, while interpretable, may miss nuanced queries. Future work will explore embedding-based semantic classification.

Personalization: Current recommendations are population-level optima. User-specific preference learning could improve individual utility.

9 Conclusion

We presented LLM Jury, an open-source library for multi-objective LLM selection. Our hybrid Pareto-Chebyshev optimization identifies models achieving 85-95% of frontier quality at 10-30% of cost, enabling significant operational savings without meaningful quality degradation.

Key technical contributions include:

- 5-dimensional Chebyshev scalarization with Pareto dominance
- Task-specific quality scoring with non-saturating distributions
- Use-case-driven optimization that matches models to task-specific requirements (latency, context, trust)
- Fully configurable parameters enabling use-case-specific tuning and A/B experimentation
- Comprehensive evaluation across 46 current-generation models

LLM Jury is available as production-ready open-source software at [URL redacted], with comprehensive documentation and visualization tools. We believe this work addresses a critical gap between academic model evaluation and practical deployment needs.

References

- [1] Artificial Analysis. LLM Performance Leaderboard. <https://artificialanalysis.ai/>, 2024.
- [2] OpenRouter. Unified LLM API. <https://openrouter.ai/>, 2024.
- [3] Chen, L., Zaharia, M., and Zou, J. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv preprint arXiv:2305.05176, 2023.
- [4] RouteLLM Team. RouteLLM: Learning to Route LLMs with Preference Data. arXiv preprint, 2024.
- [5] Martian. Intelligent Model Routing. <https://withmartian.com/>, 2024.
- [6] Unify AI. LLM Routing Platform. <https://unify.ai/>, 2024.
- [7] Liang, P., et al. Holistic Evaluation of Language Models. arXiv preprint arXiv:2211.09110, 2022.

- [8] Hugging Face. Open LLM Leaderboard. https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard, 2023.
- [9] Zheng, L., et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv preprint arXiv:2306.05685, 2023.
- [10] Miettinen, K. Nonlinear Multiobjective Optimization. Springer, 1999.
- [11] Elsken, T., Metzen, J.H., and Hutter, F. Neural Architecture Search: A Survey. JMLR, 20(55):1-21, 2019.
- [12] Karl, F., et al. Multi-Objective Hyperparameter Optimization. AutoML Conference, 2022.
- [13] Das, I. and Dennis, J.E. A Closer Look at Drawbacks of Minimizing Weighted Sums of Objectives. Structural Optimization, 14(1):63-69, 1997.
- [14] Bowman, V.J. On the Relationship of the Tchebycheff Norm and the Efficient Frontier of Multi-Criteria Objectives. Lecture Notes in Economics and Mathematical Systems, 1976.
- [15] Branke, J., et al. Finding Knees in Multi-objective Optimization. PPSN VIII, 2004.
- [16] Kahneman, D. and Tversky, A. Prospect Theory: An Analysis of Decision under Risk. Econometrica, 47(2):263-291, 1979.
- [17] Vectara. Hallucination Leaderboard. <https://github.com/vectara/hallucination-leaderboard>, 2024.
- [18] Khattab, O., et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. arXiv preprint arXiv:2310.03714, 2023.
- [19] LangChain. LangChain: Building Applications with LLMs. <https://langchain.com/>, 2023.

A Benchmark Descriptions

Table 9 provides detailed descriptions of the 12 benchmarks used in quality scoring.

B API Usage Example

```
from llm_jury import (
    get_value_recommendations,
    classify_prompt,
    OptimizationStrategy,
    list_available_models
)

# Use bundled model cache (default)
models = list_available_models()
print(f"Available: {len(models)} models")

# Or use custom/updated cache file
models = list_available_models(
    cache_path="path/to/my_updated_cache.json"
)
```


Table 9: Benchmark descriptions

Benchmark	Description
intelligence_index	Composite general reasoning ability
coding_index	Programming across difficulty levels
math_index	Mathematical reasoning and calculation
mmlu_pro	Massive Multitask Language Understanding (Pro)
gpqa	Graduate-level science questions
hle	Hard Language understanding Evaluation
livecodebench	LeetCode-style algorithmic problems
scicode	Scientific/research-level coding
math_500	Competition mathematics problems
aime	American Invitational Math Exam
trust_score	100 - hallucination_rate
context_score	Log-normalized context window size

```

# Classify prompt to optimal use case
result = classify_prompt(
    "Write a Python function to parse JSON"
)
print(f"Use case: {result.use_case}")
# Output: code_generation

# Get value-optimized recommendations
recs = get_value_recommendations(
    baseline_model="Gemini 2.5 Pro",
    strategy=OptimizationStrategy.HYBRID,
    min_quality_pct=85,
    max_cost_pct=30,
    top_n=5,
    # cache_path="custom_cache.json" # optional
)

for rec in recs:
    print(f"{rec.model_name}: {rec.score:.2f}")

```

Quality Score Distribution: Open Source vs Proprietary Models

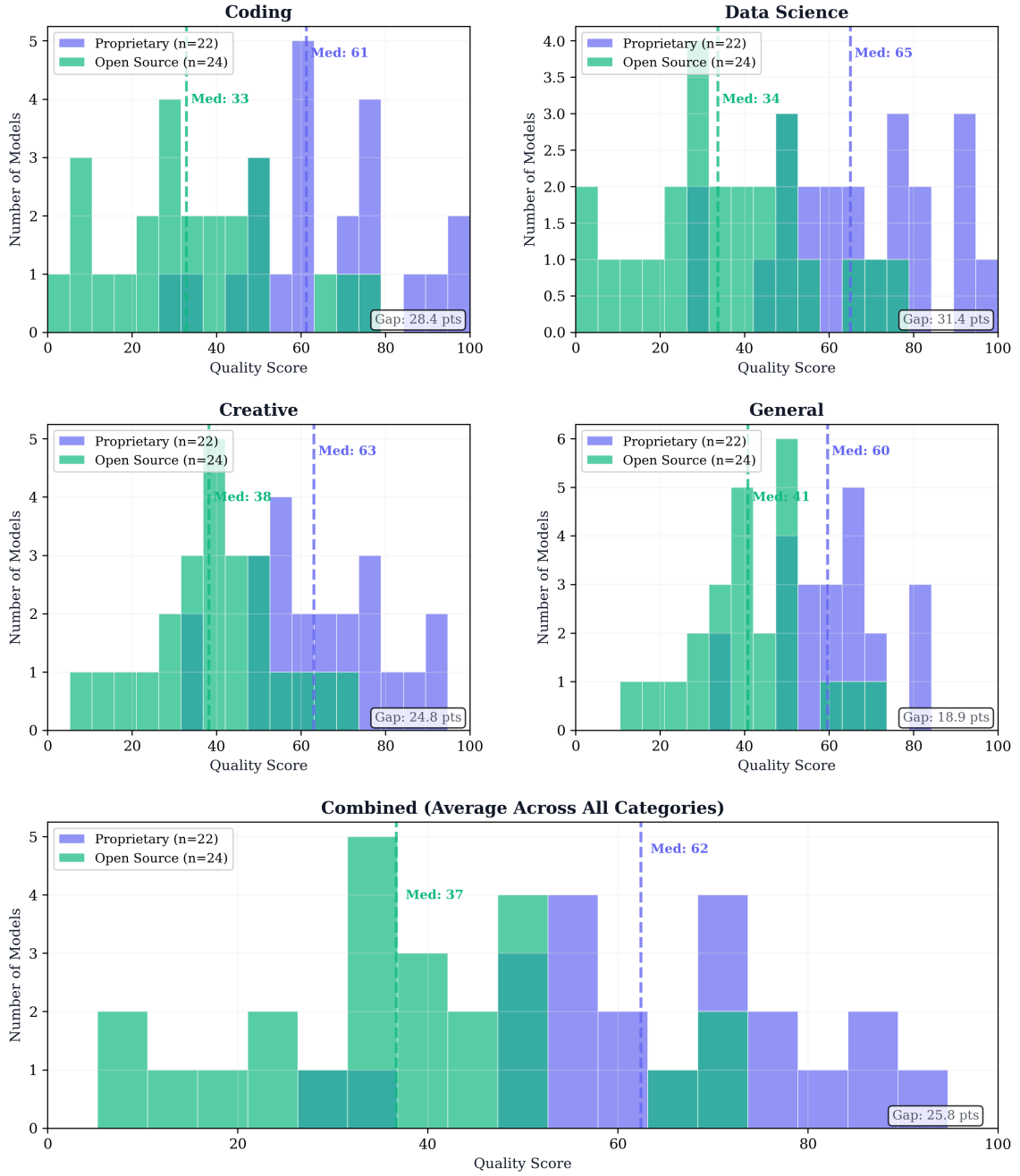


Figure 5: Quality score distributions by license type across four task categories (top two rows) and combined average (bottom). Median scores shown as dashed lines. Open-source models (green) show the smallest gap in GENERAL tasks and largest in CREATIVE tasks.